

Dokumentasi Praktikum Pemrograman Mobile

Pertemuan 3



Nama:

Muhamad Rizqi Candra

NIM:

20240801035

UNIVERSITASA ESA UNGGUL TEKNIK INFORMATIKA

FAKULTAS ILMU KOMPUTER

2026

1. Pendahuluan

Pada praktikum pertemuan ketiga ini saya mempelajari cara menerima input dari pengguna menggunakan `TextField` dan `TextEditingController`. Selain itu, saya juga mencoba membuat Form dan `TextFormField` supaya input yang dimasukkan bisa diperiksa terlebih dahulu sebelum diproses oleh aplikasi. Di praktikum ini juga mulai dibahas state, yaitu data yang nilainya bisa berubah dan dapat memengaruhi tampilan aplikasi.

Untuk aplikasi yang masih sederhana, perubahan data bisa ditangani dengan `setState`. Tetapi kalau data yang sama harus digunakan di beberapa widget atau halaman, cara tersebut akan menjadi lebih sulit untuk dikelola. Karena itu, pada praktikum ini saya menggunakan `ChangeNotifier` dan `Provider` untuk membantu mengatur state aplikasi.

2. Tujuan Praktikum

- Memahami cara menggunakan `TextField` dan `TextEditingController`.
- Membuat form yang memiliki validasi pada input.
- Memahami perbedaan penggunaan `setState` dengan state management.
- Mencoba menerapkan `ChangeNotifier` dan `Provider` pada aplikasi.
- Memahami penggunaan `context.watch()` dan `context.read()`.

3. Teori dasar

- `TextField` digunakan untuk menerima input berupa teks, sedangkan `TextEditingController` digunakan untuk mengambil atau mengatur nilai dari input tersebut. Kalau ada beberapa input yang perlu diperiksa sekaligus, input bisa dimasukkan ke dalam Form menggunakan `GlobalKey<FormState>`, kemudian validasinya dijalankan dengan `validate()`.
- State adalah data pada aplikasi yang nilainya dapat berubah dan perubahan tersebut bisa memengaruhi tampilan.
- `Provider` digunakan untuk menyediakan objek state, dalam hal ini objek yang menggunakan `ChangeNotifier`, agar bisa dipakai oleh widget yang membutuhkannya. Saat data berubah dan `notifyListeners()` dipanggil, widget yang sedang mendengarkan perubahan tersebut akan memperbarui tampilannya.

4. Langkah praktikum

Pertama saya membuat project Flutter baru melalui terminal dengan perintah berikut:

```
flutter create praktikum3
cd praktikum3
flutter run
```

5. Implementasi textfield

Pada bagian pertama, saya membuat halaman input sederhana untuk memasukkan nama. Teks yang diketik disimpan menggunakan `TextEditingController`. Setelah tombol ditekan, nama tersebut digunakan untuk menampilkan sapaan. Karena nilai `_salam` berubah setelah tombol ditekan, halaman ini menggunakan `StatefulWidget`.

```
class _InputPageState extends State<InputPage> {
  final _controller = TextEditingController();
  String _salam = '';

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }
}
```

Variabel `_controller` digunakan untuk mengambil teks yang dimasukkan melalui `TextField`. Sementara `_salam` digunakan untuk menyimpan teks sapaan yang nantinya ditampilkan di halaman. Saya juga menggunakan method `dispose()` supaya `TextEditingController` dilepas ketika halaman sudah tidak digunakan lagi.

Setelah itu saya membuat bagian input nama menggunakan `TextField` seperti pada kode berikut:

```
TextField(
  controller: _controller,
  decoration: const InputDecoration(
    labelText: 'Nama',
    border: OutlineInputBorder(),
  ),
)
```

Pada bagian ini `TextField` dihubungkan dengan `_controller`. `labelText` digunakan untuk memberi keterangan pada kolom input, sedangkan `OutlineInputBorder` digunakan untuk membuat tampilan garis pada kolom tersebut.

6. Implementasi form dengan validasi

Selanjutnya saya membuat Form dengan beberapa input, yaitu nama lengkap, email, jurusan, dan checkbox persetujuan. Form menggunakan `GlobalKey<FormState>` supaya semua input bisa divalidasi ketika tombol Daftar ditekan.

```
class _FormPageState extends State<FormPage> {
  final _formKey = GlobalKey<FormState>();
  final _nama = TextEditingController();
  final _email = TextEditingController();

  String? _jurusan;
  bool _setuju = false;
}
```

- _formKey digunakan untuk mengakses dan melakukan validasi pada Form.
- _nama digunakan untuk mengontrol input nama.
- _email digunakan untuk mengontrol input email.
- _jurusan digunakan untuk menyimpan pilihan jurusan.
- _setuju digunakan untuk menyimpan status checkbox persetujuan.

Kalau semua input sudah memenuhi validasi, aplikasi akan menampilkan SnackBar yang berisi nama dan jurusan yang dimasukkan.

```
TextFormField(
  controller: _nama,
  decoration: const InputDecoration(
    labelText: 'Nama Lengkap',
    border: OutlineInputBorder(),
  ),
  validator: (v) =>
    (v == null || v.trim().isEmpty)
      ? 'Nama wajib diisi'
      : null,
),
```

Untuk validasi nama, saya mengecek apakah input masih kosong atau tidak. Jika kosong, maka muncul pesan "Nama wajib diisi". Jika sudah diisi, validator mengembalikan null sehingga tidak ada pesan error.

```

TextFormField(
  controller: _email,
  keyboardType: TextInputType.emailAddress,
  decoration: const InputDecoration(
    labelText: 'Email',
    border: OutlineInputBorder(),
  ), InputDecoration
  validator: (v) {
    if (v == null || !v.contains('@')) {
      return 'Email tidak valid';
    }
    return null;
  }
)

```

Pada validasi email, pengecekan yang digunakan masih sederhana, yaitu memastikan terdapat karakter @ pada input. Jika karakter tersebut tidak ada, aplikasi akan menampilkan pesan "Email tidak valid".

Hasil akhir:

7. Mengapa butuh State Management?

Pada bagian sebelumnya, state masih berada di dalam satu halaman dan dikelola menggunakan `setState`. Untuk contoh aplikasi daftar tugas, ada halaman yang menampilkan daftar tugas dan ada halaman lain untuk menambahkan tugas. Kedua halaman tersebut perlu menggunakan data yang sama. Kalau data terus dikirim melalui constructor dan callback, pengelolaannya bisa menjadi cukup rumit. Karena itu, data dapat ditempatkan pada satu objek yang bisa digunakan bersama oleh beberapa halaman. Di sinilah Provider digunakan.

8. Implementasi provider

Sebelum membuat bagian Provider, saya terlebih dahulu menambahkan package provider melalui terminal pada folder project dengan perintah berikut:

```
flutter pub add provider
```

```
PS C:\Users\LENOVO LOQ\pemmob\pert3\praktikum3> flutter pub add provider
Resolving dependencies...
Downloading packages...
  cupertino_icons 1.0.9 (2.0.0 available)
  material_color_utilities 0.13.0 (0.13.1 available)
  meta 1.18.3 (1.19.0 available)
+ nested 1.0.0
+ provider 6.1.5+1
  test_api 0.7.12 (0.7.14 available)
```

```
class TugasModel extends ChangeNotifier {
  final List<Tugas> _items = [];

  List<Tugas> get items => List.unmodifiable(_items);

  int get jumlahSelesai =>
    _items.where((t) => t.selesai).length;
}
```

Pada bagian ini, `items` digunakan untuk menyimpan daftar tugas, sedangkan `jumlahSelesai` digunakan untuk menghitung jumlah tugas yang sudah selesai. Class ini menggunakan `ChangeNotifier` sehingga perubahan data bisa diberitahukan kepada widget yang menggunakan model tersebut.

```

void tambah(String judul) {
    _items.add(Tugas(judul));
    notifyListeners();
}

void toggle(int index) {
    _items[index].selesai = !_items[index].selesai;
    notifyListeners();
}

void hapus(int index) {
    _items.removeAt(index);
    notifyListeners();
}
}

```

Setiap method yang mengubah data memanggil notifyListeners(). Dengan begitu, widget yang menggunakan data dari model bisa mengetahui bahwa ada perubahan dan memperbarui tampilannya.

tambah() : Menambahkan tugas

toggle() : Mengubah status tugas menjadi selesai atau belum selesai

hapus() : Menghapus tugas

notifyListeners() : Memberitahu widget bahwa data pada model berubah

```

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => TugasModel(),
      child: const MyApp(),
    ), ChangeNotifierProvider
  );
}

```

ChangeNotifierProvider digunakan untuk menyediakan TugasModel supaya bisa diakses oleh halaman-halaman yang ada di dalam aplikasi.

```
void _simpan() {
  final judul = _controller.text.trim();

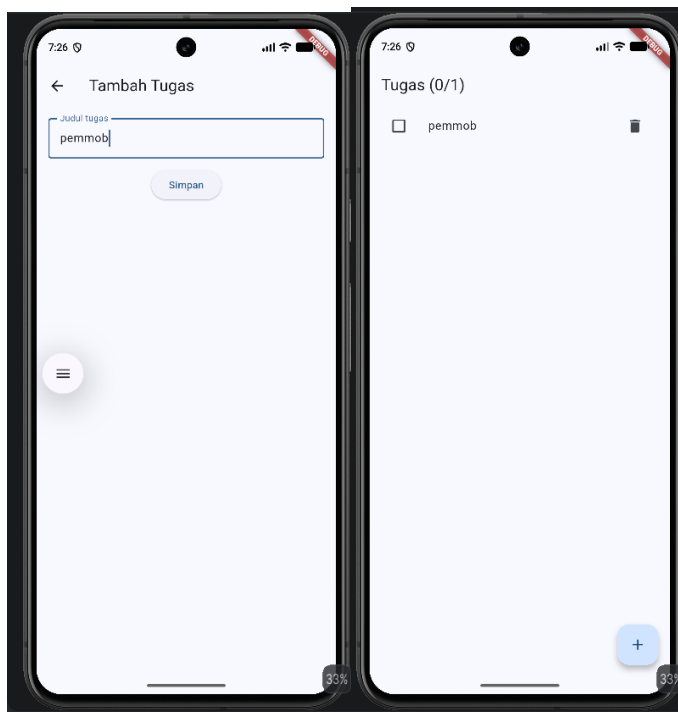
  if (judul.isEmpty) return;

  context.read<TugasModel>().tambah(judul);

  Navigator.pop(context);
}
```

- Mengambil teks yang dimasukkan dari input.
- Memeriksa agar judul tugas tidak kosong.
- Menambahkan data tugas melalui TugasModel.
- Kembali ke halaman daftar setelah tugas berhasil disimpan.

Hasil akhir:



9. Latihan Mandiri

- Menambahkan validasi pada TambahPage agar judul tugas minimal terdiri dari 3 karakter dan menampilkan pesan error jika belum memenuhi ketentuan.

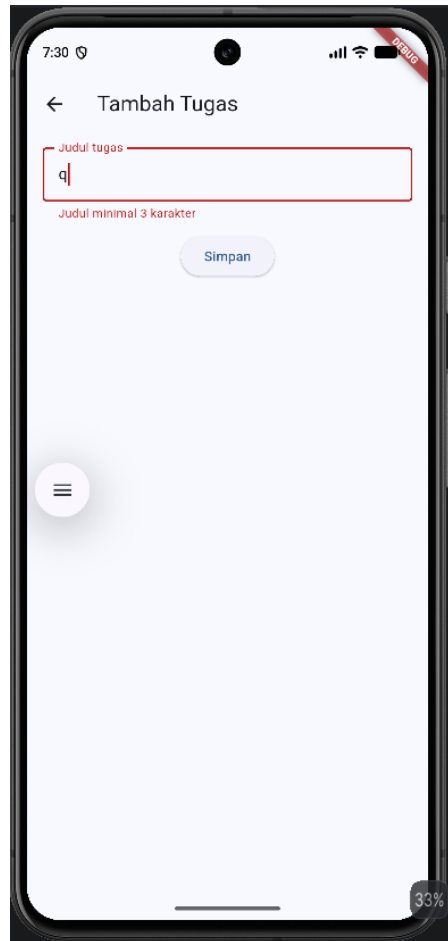
```

body: Form(
  key: _formKey,
  child: Padding(
    padding: const EdgeInsets.all(16),
    child: Column(
      children: [
        TextFormField(
          controller: _controller,
          autofocus: true,
          decoration: const InputDecoration(
            labelText: 'Judul tugas',
            border: OutlineInputBorder(),
          ),
          validator: (v) {
            if (v == null || v.trim().length < 3) {
              return 'Judul minimal 3 karakter';
            }

            return null;
          },
        ),
      ],
    ),
  ),
)

```

Pada bagian ini saya menambahkan validator untuk mengecek input judul. Jika input kosong atau jumlah karakternya kurang dari 3, akan muncul pesan "Judul minimal 3 karakter" di bawah kolom input. Jika sudah memenuhi ketentuan, validator mengembalikan null sehingga tidak ada pesan error. Hasilnya dapat dilihat pada gambar berikut:



- b. Menambahkan method `hapusSelesai()` pada `TugasModel` dan tombol pada `AppBar` untuk menghapus semua tugas yang sudah selesai.

```
void hapusSelesai() {  
    _items.removeWhere((t) => t.selesai);  
    notifyListeners();  
}
```

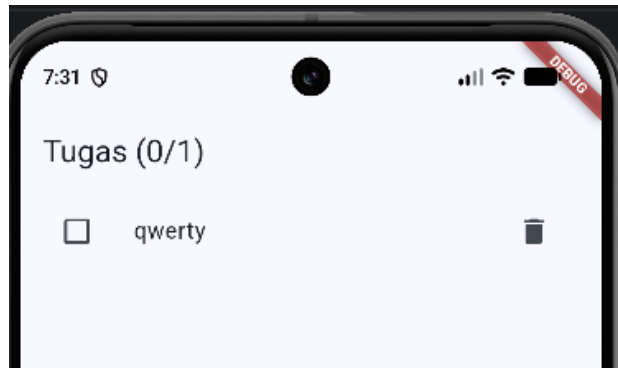
Method `hapusSelesai()` menggunakan `removeWhere` untuk mencari dan menghapus semua tugas yang statusnya sudah selesai. Setelah proses penghapusan selesai, `notifyListeners()` dipanggil agar perubahan langsung terlihat pada tampilan aplikasi.

```

appBar: AppBar(
  title: Text(
    'Tugas (${model.jumlahSelesai}/${model.items.length})',
  ),
  actions: [
    IconButton(
      icon: const Icon(Icons.delete_sweep),
      onPressed: () {
        context.read<TugasModel>().hapusSelesai();
      },
    ),
  ],
),
),
AppBar

```

Pada AppBar ditambahkan tombol di bagian kanan atas. Saat tombol tersebut ditekan, `context.read<TugasModel>().hapusSelesai()` akan dipanggil untuk menghapus semua tugas yang sudah selesai.



Jadi, jika ada beberapa tugas yang sudah ditandai selesai, saya bisa menghapus semuanya sekaligus melalui tombol tong sampah yang ada di bagian atas.

- c. Menampilkan Snackbar dengan tulisan "Tugas ditambahkan" setelah tugas berhasil disimpan.

```

void _simpan() {
  if (_formKey.currentState!.validate()) {
    final judul = _controller.text.trim();

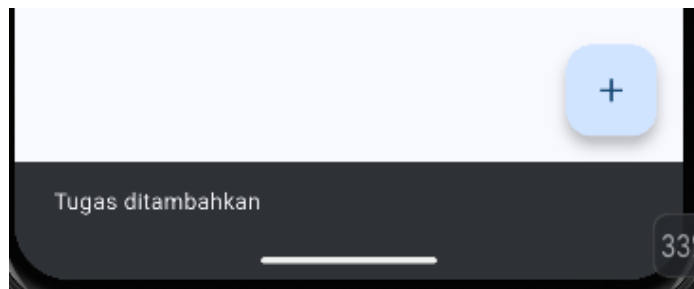
    context.read<TugasModel>().tambah(judul);

    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(
        content: Text('Tugas ditambahkan'),
      ), SnackBar
    );

    Navigator.pop(context);
  }
}

```

Setelah tugas baru berhasil ditambahkan, SnackBar akan muncul sebagai pemberitahuan bahwa tugas sudah ditambahkan. Setelah itu Navigator.pop() digunakan untuk kembali ke halaman sebelumnya. Hasilnya dapat dilihat pada gambar berikut:



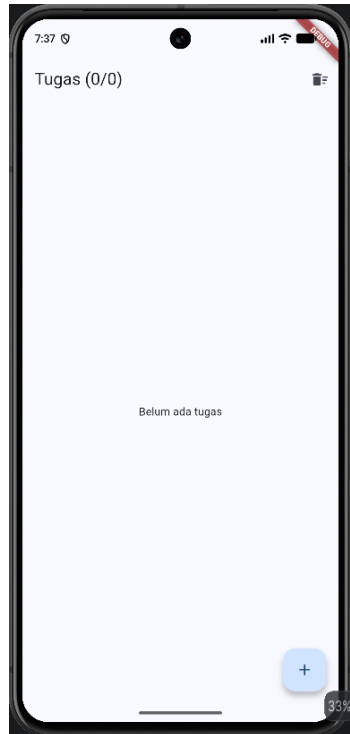
- d. Jika daftar tugas masih kosong, menampilkan tulisan "Belum ada tugas" di bagian tengah layar.

```

body: model.items.isEmpty
  ? const Center(
    child: Text('Belum ada tugas'),
  ) Center
: ListView.builder(

```

Pada kondisi daftar masih kosong, aplikasi menampilkan tulisan "Belum ada tugas" di tengah halaman. Pesan tersebut akan hilang setelah saya menambahkan tugas baru.



10. Pada bagian selanjutnya saya membuat aplikasi Daftar Belanja dengan beberapa ketentuan berikut:
- Halaman Form Tambah berisi nama barang yang wajib diisi, jumlah yang wajib berupa angka lebih dari 0, dan kategori yang dipilih melalui dropdown. Setiap input memiliki validasi.
 - Halaman Daftar menampilkan semua barang yang sudah ditambahkan. Barang dapat diberi tanda sebagai "sudah dibeli" dan juga dapat dihapus.
 - State disimpan dalam satu ChangeNotifier dan dibagikan menggunakan Provider. Pada AppBar juga ditampilkan jumlah barang yang belum dibeli.

11. Implementasi Tugas

```

class BelanjaModel extends ChangeNotifier {
    final List<Barang> _items = [];

    List<Barang> get items => List.unmodifiable(_items);

    int get jumlahBelumDibeli =>
        _items.where((barang) => !barang.sudahDibeli).length;

    void tambah(String nama, int jumlah, String kategori) {

```

BelanjaModel digunakan sebagai tempat untuk mengelola data pada aplikasi Daftar Belanja. Model ini menggunakan ChangeNotifier untuk menyimpan daftar barang yang dimasukkan pengguna. Variabel items digunakan untuk mengambil daftar barang, sedangkan jumlahBelumDibeli digunakan untuk menghitung jumlah barang yang belum ditandai sebagai sudah dibeli.

```

void tambah(String nama, int jumlah, String kategori) {
    _items.add(
        Barang(
            nama,
            jumlah,
            kategori,
        ),
    );
    notifyListeners();
}

void toggle(int index) {
    _items[index].sudahDibeli =
        !_items[index].sudahDibeli;

    notifyListeners();
}

void hapus(int index) {
    _items.removeAt(index);
    notifyListeners();
}
}

```

Bagian ini berisi beberapa fungsi untuk mengelola daftar belanja. Fungsi `tambah()` digunakan untuk menambahkan barang, `toggle()` untuk mengubah status barang menjadi sudah atau belum dibeli, dan `hapus()` untuk menghapus barang. Setelah data berubah, `notifyListeners()` dipanggil supaya tampilan aplikasi ikut diperbarui.

```

runApp(
    ChangeNotifierProvider(
        create: (_) => BelanjaModel(),
        child: const MyApp(),
    ), ChangeNotifierProvider
);
}

```

Provider digunakan untuk menyediakan `BelanjaModel` agar bisa diakses oleh beberapa halaman dan widget. Dengan cara ini, state tidak perlu dibuat dan dikelola secara terpisah di setiap halaman.

```

class _TambahBarangPageState
    extends State<TambahBarangPage> {
    final _formKey = GlobalKey<FormState>();

    void _simpan() {
        if (_formKey.currentState!.validate()) {
            context.read<BelanjaModel>().tambah(
                _nama.text.trim(),
                int.parse(_jumlah.text),
                _kategori!,
            );

            Navigator.pop(context);
        }
    }
}

```

Bagian ini digunakan untuk memproses data yang berasal dari form tambah barang. `GlobalKey<FormState>` digunakan untuk menjalankan validasi setiap input. Jika semua input sudah benar, data dikirim ke `BelanjaModel` melalui fungsi `tambah()`, kemudian halaman form ditutup menggunakan `Navigator.pop()`. Untuk jumlah barang, validasi memastikan input tidak kosong, harus berupa angka, dan nilainya lebih dari 0.

```

Widget build(BuildContext context) {
  final model = context.watch<BelanjaModel>();

  return Scaffold(
    appBar: AppBar(
      title: Text(
        'Belum Dibeli (${model.jumlahBelumDibeli}}',
      ),
    ),
    body: ListView.builder(
      itemCount: model.items.length,
      itemBuilder: (context, index) {
        final barang = model.items[index];

        return ListTile(
          leading: Checkbox(
            value: barang.sudahDibeli,
            onChanged: (_) {
              context.read<BelanjaModel>().toggle(index);
            },
          ),
          title: Text(
            barang.nama,
            style: TextStyle(
              decoration: barang.sudahDibeli
                ? TextDecoration.lineThrough
                : null,
            ),
          ),
        );
      },
    ),
  );
}

```

Bagian ini digunakan untuk menampilkan seluruh barang yang sudah tersimpan di BelanjaModel. `context.watch()` digunakan supaya halaman dapat mendengarkan perubahan data dan otomatis memperbarui tampilan. Checkbox digunakan untuk memberi tanda bahwa barang sudah dibeli, sedangkan `context.read()` digunakan untuk menjalankan fungsi `toggle()` ketika checkbox diubah.

12. Hasil Tugas



7:44

← Tambah Barang

Nama barang
bebek

Jumlah
3

Kategori
Makanan

Simpan

DEBUG



7:44

← Tambah Barang

Nama barang
bebek

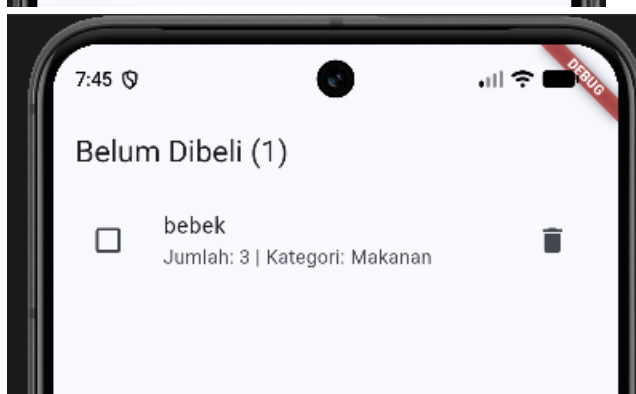
Jumlah
q

Jumlah harus angka lebih dari 0

Kategori
Makanan

Simpan

DEBUG



7:45

Belum Dibeli (1)

☐ bebek
Jumlah: 3 | Kategori: Makanan

DEBUG

13. Pertanyaan Refleksi

- Mengapa `TextEditingController` harus di-dispose?
`TextEditingController` perlu di-dispose karena controller menggunakan resource dan memiliki listener untuk memantau perubahan pada teks dan posisi kursor. Jika tidak di-dispose ketika halaman sudah tidak digunakan, resource dan listener tersebut tetap bisa tersimpan dan tidak dilepas dengan baik.
- Kapan cukup menggunakan `setState` dan kapan menggunakan `Provider`?
`setState` cukup digunakan ketika state hanya diperlukan pada satu widget atau satu halaman dan tidak perlu dibagikan ke bagian lain. Sedangkan `Provider` lebih cocok ketika data yang sama perlu digunakan oleh beberapa widget atau halaman.
- Apa yang terjadi jika `notifyListeners()` tidak dipanggil?
Kalau `notifyListeners()` tidak dipanggil setelah data pada model berubah, UI tidak akan mengetahui perubahan tersebut. Data di dalam model sebenarnya sudah berubah, tetapi widget yang menggunakan data tersebut masih bisa menampilkan kondisi sebelumnya karena tidak mendapatkan pemberitahuan untuk melakukan rebuild.
- Mengapa `onPressed` menggunakan `context.read()` bukan `context.watch()`?
`onPressed` merupakan callback yang digunakan untuk menjalankan sebuah aksi, misalnya menambah atau menghapus data. Karena pada bagian tersebut saya hanya perlu memanggil method pada model dan tidak perlu memantau perubahan state, `context.read()` lebih sesuai. Sementara `context.watch()` digunakan ketika widget perlu mendengarkan perubahan data dan melakukan rebuild secara otomatis.

14. Kesimpulan

Dari praktikum ini saya jadi lebih memahami penggunaan `TextField`, `TextEditingController`, `Form`, dan validasi input pada Flutter. Selain itu, saya juga mencoba menggunakan `ChangeNotifier` dan `Provider` untuk mengelola state yang digunakan di beberapa halaman. Pada aplikasi Daftar Belanja, data dapat disimpan dalam satu model sehingga perubahan data bisa langsung ditampilkan pada halaman yang menggunakan data tersebut.